New Horizons®
Centro de Capacitación

Curso

Desarrollo Full Stack con C# y Angular

Docente: Edwin Barrientos



New Horizons®
Centro de Capacitación

Sesión 10 — Módulo 3:

Formularios Reactivos y Validaciones



Objetivo del módulo: implementar captura de datos robusta con Reactive Forms, construyendo formularios maestro-detalle con FormBuilder y FormArray, cálculo automático de totales y validaciones cruzadas alineadas al backend C#.



New Horizons®
Centro de Capacitación

Agenda

- Reactive Forms en Angular 18: FormGroup, FormControl y FormBuilder
- Validación sincrónica y asincrónica: Reglas estándar y validadores custom
- Feedback de error en Material: mat-error, estados touched y dirty
- Maestro-detalle dinámico con FormArray: Partidas de gasto dinámicas
- Cálculo en tiempo real con valueChanges y alineación con FluentValidation



New Horizons®
Centro de Capacitación

Sesión 1:

Formularios Reactivos y Validaciones Avanzadas

¿Por qué Reactive Forms en proyectos empresariales?

New Horizons®
Centro de Capacitación



Imagina tu formulario como un avión. Con Template-driven, el piloto vuela “a ojo” desde la cabina (el HTML). Con Reactive Forms tienes la **torre de control en TypeScript**: ve cada campo, cada regla y cada cambio en tiempo real.



Modelo Explícito

El árbol de controles (FormGroup/FormArray) está desacoplado del DOM.

Piénsalo así: *Es el plano de la casa: existe antes de levantar las paredes del HTML.*

Inmutabilidad

Cada cambio emite eventos reactivos auditables y testeables.

Piénsalo así: *Es la caja negra del avión: todo cambio queda registrado y se puede reproducir en un test.*

Espejo del Backend

Replicamos los requisitos de FluentValidation en la entrada del cliente.

Piénsalo así: *Es la aduana doble: la misma regla en el aeropuerto de salida y en el de llegada.*

Jerarquía de Reactive Forms

New Horizons
Centro de Capacitación

AbstractControl

El ADN común: value · status · errors · valueChanges

FormControl

El colaborador

Rastrea el valor y la validez de **un solo campo**: `motivo`, `precio`.

FormGroup

El área

Agrupa controles. **Si un colaborador falla, el área entera falla** (validez acumulativa).

FormArray

La cuadrilla

Lista de tamaño variable: hoy 2 partidas de gasto, mañana 10.

FormBuilder

Recursos Humanos

Servicio inyectable que **contrata a todos rápido**:

```
fb.control()
fb.group({...})
fb.array([])
```

Menos `new`, menos ruido, misma estructura.

La validez sube como burbuja: si un FormControl es inválido, su FormGroup y el formulario completo también lo son. Por eso basta con mirar `formulario.invalid`.

Creación del formulario con FormBuilder

New Horizons
Centro de Capacitación

solicitud-form.ts

```ts
import { Component, OnInit, inject } from "@angular/core";
import { FormBuilder, FormGroup, Validators,
         ReactiveFormsModule } from "@angular/forms";

export class SolicitudFormComponent implements OnInit {
  private fb = inject(FormBuilder);
  formulario!: FormGroup;

  ngOnInit(): void {
    this.formulario = this.fb.group({
      motivo: ["", [Validators.required,
                    Validators.minLength(10)]],
      centroCostoId: [null, [Validators.required]],
      solicitanteId: [1],
      detalles: this.fb.array([])
    });
  }
}
```



FormBuilder es tu chef

Tú dictas la receta (un objeto) y él arma la bandeja con todos los controles listos.

1 inject(FormBuilder)

Contratas al chef sin constructor: inyección moderna de Angular.

2 fb.group({ ... })

La bandeja: un FormGroup con los campos de la solicitud.

3 [valor, [reglas]]

Cada campo es una dupla: valor inicial + lista de validadores.

4 fb.array([])

Bandeja vacía para las partidas: crecerá en tiempo de ejecución.

Validadores sincrónicos nativos y custom

New Horizons®
Centro de Capacitación

custom-validator.ts

```ts
import { AbstractControl, ValidationErrors, ValidatorFn }
  from "@angular/forms";

// Validador personalizado: al menos N palabras descriptivas
export function motivoMinimoPalabras(min: number): ValidatorFn {
  return (control: AbstractControl): ValidationErrors | null => {
    if (!control.value) return null;                          // 1
    const palabras = control.value.trim().split(/\s+/);       // 2
    return palabras.length >= min                             // 3
      ? null                                                  // 4
      : { motivoInsuficiente: true };
  };
}

// Uso:
motivo: ["", [Validators.required, motivoMinimoPalabras(3)]]
```

El portero de la discoteca

Mira al control y responde: “pasa” (`null`) o “no pasa” (`{ error }`).

Banco de pruebas · motivoMinimoPalabras(3)

⛔ **“Compra”**
1 palabra → `{ motivoInsuficiente }`

✅ **“Compra de laptops para TI”**
5 palabras → `null`

💡 **Un validador, una responsabilidad.** ① Si está vacío devuelve `null`: de eso se encarga `required`. ② `split(/\s+/)` ignora espacios dobles.

Visualización de errores con mat-error

New Horizons®
Centro de Capacitación

mat-error-view.html

```
<!-- mat-error reacciona a los estados touched e invalid -->
<mat-form-field appearance="outline" class="full-width">
  <mat-label>Motivo del Reembolso o Gasto</mat-label>
  <textarea matInput formControlName="motivo" rows="3"></textarea>
1 <mat-error *ngIf="formulario.get('motivo')?.hasError('required')">
    El motivo es obligatorio.
  </mat-error>
2 <mat-error *ngIf="formulario.get('motivo')?.hasError('minlength')">
    Debe contener al menos 10 caracteres.
  </mat-error>
</mat-form-field>
```

El campo es un semáforo

Gris mientras nadie lo toca; rojo cuando el usuario sale del campo y la regla falla.

Así lo ve el usuario

Motivo del Reembolso o Gasto

Compra|

Debe contener al menos 10 caracteres.

1 **hasError('required')**

Campo vacío → “El motivo es obligatorio.”

2 **hasError('minlength')**

Muy corto → “al menos 10 caracteres.”

El poder de FormArray para maestro-detalle

New Horizons
Centro de Capacitación

FormArray es un tren: la solicitud es la locomotora y cada partida de gasto es un vagón que enganchas o sueltas en tiempo de ejecución.



`push()`

Engancha un vagón nuevo.

La fila llega con sus validadores intactos.

`removeAt(i)`

Desengancha el vagón i.

Angular recalcula la validez global al instante.

`controls`

Recorre el tren completo.

Arreglo iterable para @for (o *ngFor) en la tabla.

Manipulación de FormArray en TypeScript

New Horizons®
Centro de Capacitación

form-array-methods.ts

```typescript
1 get detalles(): FormArray {
    return this.formulario.get("detalles") as FormArray;
  }

2 agregarFila(): void {
    this.detalles.push(this.fb.group({
      concepto: ["", [Validators.required, Validators.minLength(3)]],
      cantidad: [1, [Validators.required, Validators.min(1)]],
      precioUnitario: [0, [Validators.required, Validators.min(0.1)]]
    }));
  }

3 eliminarFila(i: number): void {
    this.detalles.removeAt(i);
  }
```



Un molde de ladrillos

Cada fila sale del mismo molde: misma forma, mismas reglas. Nunca una fila “a mano”.

1 El getter = acceso directo

Evita repetir `get('detalles') as FormArray` en el TS y en el template.

2 agregarFila() = el molde

Cada `push` crea un FormGroup nuevo con concepto, cantidad y precio validados.

3 eliminarFila(i) = desenganchar

`removeAt(i)` saca la fila y la validez del formulario se recalcula sola.

Renderizado dinámico de partidas de gasto

New Horizons
Centro de Capacitación

form-array.html

```
<!-- Iteración con formArrayName y [formGroupName] -->
<div formArrayName="detalles">
  @for (item of detalles.controls; track $index; let i = $index) {
    <div [formGroupName]="i" class="item-row">
      <input formControlName="concepto" placeholder="Concepto">
      <input type="number" formControlName="cantidad">
      <input type="number" formControlName="precioUnitario">
      <span>{{ calcularSubtotal(i) | currency:'PEN' }}</span>
      <button (click)="eliminarFila(i)">Eliminar</button>
    </div>
  }
</div>
```

Una sala de cine

Para encontrar un asiento necesitas sala, fila y butaca. El template funciona igual.

1 formArrayName = la sala

El contenedor de todas las partidas.

2 [formGroupName]="i" = la fila

El índice de cada partida dentro del arreglo.

3 formControlName = la butaca

El campo exacto: concepto, cantidad o precio.

Fila	Concepto	Cant.	Subtotal
0	Taxi aeropuerto	2	**S/ 90.00**
1	Almuerzo cliente	1	**S/ 120.00**

Cálculo en tiempo real con valueChanges

New Horizons
Centro de Capacitación

calculo-total.ts

```ts
get montoTotalCalculado(): number {
  return this.detalles.controls.reduce((acc, ctrl) => {
    const c = ctrl.get("cantidad")?.value || 0;
    const p = ctrl.get("precioUnitario")?.value || 0;
    return acc + (c * p);
  }, 0);
}

// El total se refresca automáticamente con cada pulsación del usuario

// Variante explícita con valueChanges (sin fugas de memoria):
this.detalles.valueChanges
  .pipe(takeUntilDestroyed(this.destroyRef))
  .subscribe(() => this.total = this.montoTotalCalculado);
```

1
2



La caja registradora del súper

Cada vez que pasa un producto, el total en pantalla se actualiza solo. Nadie presiona “calcular”.

2 × Taxi aeropuerto	S/ 90.00
1 × Almuerzo cliente	S/ 120.00
3 × Útiles de oficina	S/ 25.50
TOTAL	**S/ 235.50**

1 **reduce = sumar fila por fila**

2 **valueChanges + takeUntilDestroyed**

Suscripción que se limpia sola al destruir el componente.

Alineación Frontend con FluentValidation C#

New Horizons
Centro de Capacitación

🔑 **Dos cerraduras, la misma llave.** Angular avisa en la puerta (UX inmediata); FluentValidation custodia la bóveda (seguridad). Si las reglas no coinciden, el usuario “pasa” la puerta y choca con un 400.

🛡 Backend · FluentValidation (C#)	Campo	📄 Frontend · Angular
`NotEmpty(), MinimumLength(10)`	⇄ Motivo	`Validators.required, minLength(10)`
`GreaterThan(0)`	⇄ Centro de Costo	`Validators.required (selección obligatoria)`
`Must(d => d.Any())`	⇄ Detalles	`FormArray con mínimo 1 fila`
`GreaterThan(0)`	⇄ Precio y Cantidad	`Validators.min(1) y Validators.min(0.1)`

Validación de cruce en formulario (Cross-Field)

New Horizons®
Centro de Capacitación

cross-field-validator.ts

```ts
// Validador a nivel de grupo completo:
export const montoMaximoValidator: ValidatorFn =
  (g: AbstractControl): ValidationErrors | null => {
    const detalles = g.get("detalles") as FormArray;
    if (!detalles) return null;
    const total = detalles.controls.reduce(
      (acc, c) => acc + c.value.cantidad * c.value.precioUnitario, 0);
    return total > 50000 ? { topeExcedido: true } : null;
  };

// Se registra en el grupo, no en un control:
this.formulario = this.fb.group({ /* ...campos... */ },
  { validators: montoMaximoValidator });
```

1
2
3

El auditor del ticket

Cada partida puede estar bien por sí sola; él revisa la suma completa.

✓	Laptops 2 × 18,500	**S/ 37,000**
✓	Monitores 4 × 2,800	**S/ 11,200**
✓	Licencias 5 × 820	**S/ 4,100**

⛔ **Total S/ 52,300 > tope S/ 50,000**

→ `{ topeExcedido: true }`

💡 Muéstralo en el template con `formulario.hasError('topeExcedido')`: el error vive en el grupo, no en un campo.

Blindaje del botón Guardar

New Horizons®
Centro de Capacitación

submit-button.html

```
<!-- El botón Guardar permanece inactivo si hay errores -->
<button mat-raised-button color="primary"
        type="submit"
1       [disabled]="formulario.invalid || detalles.length === 0">
  <mat-icon>save</mat-icon> Guardar Solicitud
</button>
```

Tabla de verdad del candado

formulario.invalid	detalles.length	Botón
true	cualquiera	🔒 Bloqueado
false	0	🔒 Bloqueado
false	≥ 1	🔓 Habilitado

Un botón con doble candado

Solo se abre si el formulario es válido **y** tiene al menos una partida.

Lo que ve el usuario

💾 Guardar	💾 Guardar
con errores	todo en orden

⚠️ **El disabled es UX, no seguridad.** Desde DevTools se puede habilitar. En `onSubmit()` vuelve a validar y llama a `markAllAsTouched()` para revelar todos los errores.

Modo Edición: patchValue() en acción

New Horizons®
Centro de Capacitación

patch-value.ts

```ts
cargarParaEditar(s: Solicitud): void {
  this.formulario.patchValue({
    motivo: s.motivo, centroCostoId: s.centroCostoId
  });
  this.detalles.clear();
  s.detalles.forEach(d => {
    this.detalles.push(this.fb.group({
      concepto: [d.concepto, Validators.required],
      cantidad: [d.cantidad, Validators.min(1)],
      precioUnitario: [d.precioUnitario, Validators.min(0.1)]
    }));
  });
}
```

1
2
3



Corregir un formulario impreso

Rellenas la cabecera y vuelves a armar las filas del detalle desde cero.

1 **patchValue = rellenar**

Solo asigna las claves que envías.

2 **clear() = vaciar el tren**

Evita filas duplicadas al editar dos veces.

3 **push() por cada partida**

Reconstruye cada fila con sus validadores.

setValue()

Examen: respondes todo o falla.

patchValue()

Borrador: completas lo que sabes.

Estados de un control: PRISTINE, DIRTY, TOUCHED

New Horizons
Centro de Capacitación

📖 **Un campo es como un libro nuevo.** No regañas a alguien por no haber leído un libro que aún no abre: los errores aparecen solo cuando el usuario ya interactuó.

Pristine / Dirty

Recién impreso vs. ya subrayado.

Pristine: sin cambios. Dirty: el usuario ya modificó el valor original.

Untouched / Touched

Nadie lo abrió vs. alguien lo hojeó y lo cerró.

Touched: el control perdió el foco (blur). Ideal para desplegar mat-error.

Valid / Invalid

¿Aprueba la revisión del editor?

Estado actualizado tras evaluar todos los validadores.

⭐ **Regla de oro:** muestra el error cuando `control.invalid && (control.touched || control.dirty)`.

El DTO resultante listo para la API

New Horizons
Centro de Capacitación

dto-ready.ts

```ts
// Body exacto que viaja en POST /api/solicitudes:
const dto: CrearSolicitudDto = {
  motivo: this.formulario.value.motivo,
  centroCostoId: this.formulario.value.centroCostoId,
  solicitanteId: 1,
  detalles: this.formulario.value.detalles
};
```

1

POST /api/solicitudes · body (JSON)

```json
{
  "motivo": "Viaje a obra Arequipa para supervisión",
  "centroCostoId": 12,
  "solicitanteId": 1,
  "detalles": [
    { "concepto": "Pasaje aéreo", "cantidad": 1, "precioUnitario": 480 }
  ]
}
```



La maleta y la aduana

El formulario es la maleta; el DTO es solo lo que pasa por la aduana de la API: nada más, nada menos.

1

Tipa con CrearSolicitudDto

Si falta o sobra una propiedad, TypeScript te avisa antes que C#.

2

Ojo con .value

Omite los controles disabled. Si los necesitas, usa `getRawValue()`.

3

Mismo contrato que el backend

Los nombres coinciden 1:1 con el DTO de C# (camelCase en JSON).

Flujo de captura de una solicitud

New Horizons
Centro de Capacitación

1

Encabezado

La carátula del expediente

- ✓ Motivo descriptivo obligatorio
- ✓ Asignación de centro de costo

Visto en: *FormBuilder · validadores · mat-error*

2

Partidas de Gasto

Los vagones del tren

- ✓ Concepto, cantidad y precio
- ✓ Cálculo en vivo de subtotal

Visto en: *FormArray · @for · totales en vivo*

3

Validación y Envío

El control de aduana

- ✓ Cero errores de validación
- ✓ Emisión del DTO hacia C#

Visto en: *Cross-field · botón blindado · DTO*

🚦 Si un paso falla, el siguiente no arranca: **el botón Guardar sigue con candado** hasta que los tres estén en verde.

Checklist de calidad en formularios reactivos

New Horizons
Centro de Capacitación

✈️ **Checklist de pre-vuelo:** un piloto no despega sin revisarla; tú no haces merge de un formulario sin ella.

- ✓ **Usa FormBuilder con `inject(FormBuilder)`.** → *Menos boilerplate y sin constructor.*
- ✓ **Condiciona mat-error a touched o dirty.** → *No saturas la pantalla de rojo al abrir.*
- ✓ **Implementa FormArray para maestro-detalle.** → *Listas de longitud variable sin hacks.*
- ✓ **Calcula totales con getters o signals.** → *El total nunca queda desactualizado.*
- ✓ **Verifica el tipo contra `CrearSolicitudDto`.** → *El backend recibe exactamente lo que espera.*

Errores comunes en Reactive Forms

New Horizons®
Centro de Capacitación

✗ Evita esto	✓ Haz esto
Usar [(ngModel)] dentro de un [formGroup]	Usa solo directivas de ReactiveFormsModule
Olvidar formControlName en un input	Mapea cada control con su formControlName
Guardar con detalles vacíos o precios negativos	Valida detalles.length >= 1 y cantidades > 0
setValue() con un objeto incompleto	patchValue() para asignaciones parciales
Mutar valores sin pasar por patchValue()	Sincroniza el guardado con el servicio de datos

Buenas prácticas de UX en captura de datos

New Horizons
Centro de Capacitación

Diseña para alguien con prisa, en el celular, entre dos reuniones, que solo quiere que le reembolsen el taxi.

Etiquetas que guían

Placeholders y mat-label claros en cada mat-form-field.

“Ej.: Taxi al aeropuerto”

Teclado correcto

type="number" en inputs numéricos activa el teclado adecuado en móviles.

Menos toques, menos errores

Borrar sin miedo

Elimina partidas con un solo clic, avisando si es la última.

“Debe quedar al menos una partida”

El total, protagonista

Resalta el total acumulado con tipografía destacada y formato de moneda.

S/ 235.50 visible siempre

Resumen de la sesión

New Horizons®
Centro de Capacitación

📜 Reactive Forms

FormGroup, FormControl y FormBuilder.

≈ La torre de control

✅ Validaciones

Validadores sincrónicos nativos y custom.

≈ El portero

💬 Feedback UI

mat-error reaccionando a touched y dirty.

≈ El semáforo

➖ FormArray

Partidas dinámicas agregadas y removidas.

≈ El tren

💰 Totales en Vivo

Cálculo reactivo con valueChanges.

≈ La caja registradora

🚀 Contrato DTO

Espejo idéntico a FluentValidation de C#.

≈ La aduana

PREGUNTAS...

New Horizons®
Centro de Capacitación

1. ¿Por qué preferimos Reactive Forms sobre Template-driven Forms en aplicaciones de negocio?
2. ¿Cómo refleja el formulario del frontend las mismas reglas que FluentValidation en C#?
3. ¿Qué es un FormArray y cómo modela listas de partidas de gasto dinámicas?
4. ¿Cómo calculamos automáticamente el monto total cuando cambia la cantidad o el precio unitario?
5. ¿Cómo evitamos memory leaks al suscribirnos a valueChanges de los controles?

New Horizons
Centro de Capacitación

Conclusiones

Control total

Reactive Forms proporciona control granular, inmutabilidad y pruebas unitarias aisladas.



Detalle dinámico

FormArray es la estructura ideal para capturar detalles dinámicos de compras o reembolsos.

Doble cerradura

Las validaciones de cliente mejoran la UX, pero nunca sustituyen a FluentValidation en el servidor.



Totales al instante

valueChanges permite calcular subtotales y totales instantáneamente con programación reactiva.



Contrato limpio

El desacoplamiento entre UI y modelo permite enviar directamente los datos como CrearSolicitudDto.

GRACIAS